

TALLER PRÁCTICO: ASINCRONÍA

Objetivo: Aplicar los conceptos de Promesas, Callbacks y Async/Await en JavaScript para resolver flujos de trabajo asíncronos y no bloqueantes.

🔧 PARTE 1: Consumo con Promesas Tradicionales (`.then` y `.catch`)

Desarrolla los siguientes ejercicios creando las promesas correspondientes y consumiéndolas utilizando la sintaxis de `.then()` y `.catch()`.

💻 Ejercicio 1: Simulador de Retiro de Cajero Automático (ATM)

1. Crea una función llamada `retirarDinero(monto)` que devuelva una **Promesa**.
2. **Lógica interna:**
 - Si el `monto` a retirar es menor o igual a `$500` (saldo simulado disponible), la promesa debe resolverse (`resolve`) con el mensaje: `"Retiro exitoso de $X. Saldo restante: $Y"`.
 - Si el `monto` es mayor a `$500`, la promesa debe rechazarse (`reject`) con el mensaje: `"Fondos insuficientes. Límite de retiro: $500"`.
 - Agrega un temporizador (`setTimeout`) para simular que la transacción tarda **1.5 segundos** en procesarse en el banco.
3. **Consumo:** Llama a la función y utiliza `.then()` y `.catch()` para imprimir los mensajes en la consola. Pruébala con un caso exitoso (ej: `$200`) y uno fallido (ej: `$600`).

🕒 Ejercicio 2: Verificación de Edad para Suscripción

1. Crea una función llamada `verificarEdad(edad)` que devuelva una **Promesa**.
2. **Lógica interna:**
 - Si la `edad` es igual o mayor a `18` años, la promesa se resuelve con el mensaje: `"Acceso concedido. Bienvenido a la plataforma"`.
 - Si la `edad` es menor a `18` años, la promesa se rechaza con el mensaje: `"Acceso denegado. Registro permitido solo para mayores de edad"`.
 - Simula un retraso de **1 segundo** para la verificación de datos.
3. **Consumo:** Consume la promesa usando `.then()` y `.catch()`, imprimiendo el resultado en consola.



🔧 PARTE 2: Consumo Moderno con `async` / `await` y `try-catch`

Desarrolla los siguientes ejercicios creando las promesas correspondientes y consumiéndolas utilizando funciones `async` acompañadas del operador `await` y control de errores `try-catch`.

🕒 Ejercicio 3: Consulta de Clima Local

1. Crea una función llamada `consultarClima(ciudad)` que devuelva una **Promesa**.
2. **Lógica interna:**
 - La promesa debe resolverse después de **1.5 segundos** devolviendo un objeto con el clima simulado (ej: `{ temperatura: "24°C", condicion: "Soleado" }`).
3. **Consumo:** Crea una función asíncrona llamada `mostrarClima(ciudad)` que consuma la promesa usando `await`, imprima la temperatura y condición de forma ordenada, y maneje posibles errores con `try-catch`.

📷 REQUISITOS DE ENTREGA (2 CAPTURAS DE PANTALLA)

Deberás presentar únicamente **dos capturas de pantalla completas** de la ejecución de tu taller:

- **Captura 1 (ATM):** Captura de pantalla de la terminal de VS Code (corriendo con `node`) o de la consola del navegador mostrando el **código y la consola de ejecución del Ejercicio 1 (Cajero ATM)**. Se debe visualizar la ejecución del caso exitoso (monto menor a
- **Captura 2 (Clima):** Captura de pantalla de la terminal de VS Code (corriendo con `node`) o de la consola del navegador mostrando el **código y la consola de ejecución del Ejercicio 3 Consulta de Clima Local**.
- **Repositorio en GitHub**
- **Capturas enviadas al grupo de WhatsApp**

